

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ
ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
Национальный исследовательский ядерный университет «МИФИ»



Институт интеллектуальных кибернетических систем

КАФЕДРА КИБЕРНЕТИКИ (№22)

Реферат

Студент гр. Б22-534

Колесников Михаил Леонидович

(ФИО)

ТЕМА РЕФЕРАТА

Сравнительная характеристика методологий разработки: RUP, OpenUP, SCRUM, V-Model, RAD (Rapid Application Development)

Москва 2025

Содержание

Введение	3
1 Описание методологий разработки ПО	5
1.1 RUP (Rational Unified Process)	5
1.2 OpenUP (Open Unified Process)	6
1.3 SCRUM	8
1.4 V-Model	10
1.5 RAD (Rapid Application Development)	11
2 Сравнительная характеристика методологий	14
2.1 Приоритеты в каждом подходе	14
2.2 Политики в отношении анализа и проектирования	16
2.3 Политики в отношении написания и организации кода	18
2.4 Политики в отношении тестирования	20
2.5 Политики в отношении внедрения	22
2.6 Организация взаимодействия внутри команды (или между командами)	23
Заключение	26
Литература	28

Введение

В современном мире информационных технологий разработка программного обеспечения представляет собой сложный процесс, требующий не только технических навыков, но и четкой организации работ, что подчеркивает важность методологий как систематических подходов к управлению проектами [1]. Эти методологии объединяют правила, принципы и практики, позволяющие структурировать этапы создания ПО, минимизировать риски и оптимизировать ресурсы, включая планирование, анализ, реализацию и поддержку системы [2]. В частности, они помогают избежать хаотичности в работе команды, обеспечивая предсказуемость результатов и адаптацию к изменяющимся требованиям заказчика, что особенно актуально в условиях быстрых технологических трендов, таких как переход к гибким методам, ориентированным на итеративность и взаимодействие [2]. Анализ и классификация этих методологий служат основой для понимания их различий, позволяя выделить ключевые критерии, такие как гибкость, фокус на рисках или качество, что способствует выбору оптимального подхода для конкретного проекта [3]. В рамках данного реферата рассматривается сравнительная характеристика пяти популярных методологий: RUP, OpenUP, SCRUM, V-Model и RAD, с акцентом на их приоритеты, такие как управление рисками, скорость разработки или обеспечение качества, а также на политики в отношении анализа требований, проектирования архитектуры, написания и организации кода, тестирования функциональности, внедрения системы и организации взаимодействия внутри команды или между несколькими командами разработчиков. Такой подход позволяет выявить сильные и слабые стороны каждой методологии, опираясь на их фундаментальные принципы и практические аспекты применения.

Актуальность темы обусловлена тем, что правильный выбор методологии напрямую влияет на успех проекта, снижая вероятность срывов сроков, перерасхода бюджета или несоответствия конечного продукта ожиданиям пользователей [4]. Например, в условиях неопределенности требований или высоких рисков неподходящая методология может привести к значительным потерям, как отмечается в анализе существующих подходов, где подчеркивается необходимость учета специфики проекта, включая его масштаб и сложность [5]. Блок-схемы и алгоритмы выбора, основанные на оценке рисков, стабильности требований и размера команды, демонстрируют, что игнорирование этих факторов часто приводит к неудачам, особенно в крупных или исследовательских проектах, где требуется баланс между формализацией и гибкостью [4]. В контексте растущего разнообразия методологий, от традиционных до agile-подходов, сравнительный анализ помогает менеджерам проектов принимать обоснованные решения, повышая эффективность разработки и адаптивность к внешним изменениям [5].

Цель реферата заключается в проведении сравнительного анализа методологий RUP, OpenUP, Scrum, V-Model и RAD по ключевым приоритетам и организационным политикам. Для достижения этой цели решаются следующие задачи: описать принципы, фазы и приоритеты RUP как итеративно-инкрементальной методологии; охарактеризовать OpenUP как облегченную версию RUP с акцентом на гибкость и минимализм; изложить основы SCRUM, включая его ценности, роли и события для обеспечения гибкости; представить структуру V-Model с фокусом на качество и раннее тестирование; раскрыть принципы RAD, ориентированные на

скорость и вовлеченность пользователей; сравнить приоритеты в каждом подходе, такие как управление рисками в RUP или клиентская ценность в SCRUM; проанализировать политики в отношении анализа и проектирования, включая фокус на требованиях и моделях; оценить подходы к написанию и организации кода с точки зрения итеративности и стандартов; изучить политики тестирования, включая уровни интеграции и верификации; сравнить стратегии внедрения, от переходных фаз до инкрементальных релизов; и, наконец, рассмотреть организацию взаимодействия внутри команды или между командами, с учетом ролей, митингов и самоорганизации.

1 Описание методологий разработки ПО

1.1 RUP (Rational Unified Process)

Rational Unified Process, или RUP, представляет собой итеративно-инкрементальную методологию разработки программного обеспечения, разработанную компанией Rational Software во второй половине 1990-х годов и впоследствии интегрированную в продукты IBM, которая подчеркивает дисциплинированный подход к управлению проектами через четкие фазы и итерации [6]. Эта методология возникла как эволюция более ранних процессов, таких как водопадная модель, но отличается от нее тем, что позволяет корректировать требования и дизайн на протяжении всего жизненного цикла проекта, минимизируя риски за счет постепенного наращивания функциональности [5]. В основе RUP лежит идея, что разработка должна быть ориентирована на архитектуру системы, с акцентом на управление рисками и постоянную оценку прогресса, что делает ее подходящей для сложных проектов, включая те, что связаны с платформами вроде System z, где требуется интеграция legacy-систем с современными технологиями [6]. Основные принципы RUP включают итеративность, когда проект делится на мини-проекты, каждый из которых проходит полный цикл от требований до тестирования, что позволяет рано выявлять проблемы и адаптироваться к изменениям [1]. Кроме того, методология подчеркивает управление рисками, где на ранних этапах идентифицируются и разрешаются ключевые угрозы, такие как неопределенность требований или технические сложности, чтобы избежать катастрофических последствий на поздних стадиях [6]. Еще одним ключевым принципом является фокус на архитектуре, где создается исполняемая архитектура системы уже в начале, позволяющая тестировать критические компоненты и корректировать дизайн timely [5]. Жизненный цикл RUP разделен на четыре фазы: inception, elaboration, construction и transition, каждая из которых может включать несколько итераций в зависимости от масштаба проекта [6]. В фазе inception основной акцент делается на определении видения проекта, границ, бизнес-кейса и ключевых прецедентов использования, с оценкой рисков и бюджета, чтобы решить, стоит ли продолжать разработку [6]. Здесь требования собираются в виде use cases, что помогает получить целостное представление о функциональности системы, в отличие от простого перечисления функций [1]. Фаза elaboration фокусируется на детализации требований, создании базовой архитектуры и снижении основных рисков, где реализуется 20-30% прецедентов для тестирования архитектурных решений, что отличает RUP от водопадной модели, где дизайн фиксируется рано без практической проверки [6]. В этой фазе используются визуальные инструменты вроде UML-диаграмм для моделирования предметной области, классов и взаимодействий, что обеспечивает ясность и возможность корректировок [5]. Construction-фаза посвящена основной разработке, где создается большая часть кода, промежуточные прототипы и проводится интеграция компонентов, с итерациями, длящимися от 2 до 6 недель, чтобы постепенно наращивать функциональность [6]. Наконец, transition включает бета-тестирование, обучение пользователей, развертывание и миграцию данных, с анализом результатов для будущих улучшений [1]. Приоритеты RUP ориентированы на архитектуру и требования: методология считает, что сильная архитектурная основа должна быть заложена рано, чтобы поддерживать изменения, а требования описываются через use cases для лучшего понимания пользовательских нужд [6].

В контексте System z RUP адаптируется для green field development и эволюции систем, включая превращение существующих возможностей в веб-сервисы, с учетом специфики COBOL и CICS, но исключая чистое обслуживание [6]. RUP также включает дисциплины, такие как бизнес-моделирование, управление требованиями, анализ и дизайн, реализация, тестирование и развертывание, где усилия распределяются динамически: в начале больше на анализ, позже на реализацию и тестирование [5]. Это делает RUP гибкой, но требующей настройки под организацию, с возможностью использования инструментов вроде Rational Method Composer для публикации и адаптации процесса [6]. В целом, RUP подходит для проектов с жесткими требованиями к качеству, таких как в оборонной или космической отраслях, где предсказуемость и контроль рисков критичны, хотя для небольших команд она может показаться тяжеловесной без правильной адаптации [1].

1.2 OpenUP (Open Unified Process)

Open Unified Process, или OpenUP, представляет собой итеративно-инкрементальный метод разработки программного обеспечения, который позиционируется как легкая и гибкая вариация Rational Unified Process, известного как RUP [7]. Этот подход возник как ответ на необходимость баланса между тяжелыми традиционными методологиями, перегруженными деталями и регламентами, и крайне облегченными agile-подходами, которые полагаются на высокую зрелость команды и часто игнорируют формальные процессы [8]. OpenUP был разработан в рамках проекта Eclipse Process Framework с участием специалистов из различных компаний, включая IBM, и сочетает в себе элементы RUP, Scrum, Extreme Programming и других методик, чтобы создать прагматичный процесс, ориентированный на коллективную работу и адаптивность [9]. В его основе лежит философия экономичного производства, заимствованная из Lean Manufacturing, которая подчеркивает достижение высокого качества результатов при минимизации потерь времени и ресурсов, а также фокус на создании ценности для заинтересованных сторон [9].

Одним из ключевых преимуществ OpenUP является его независимость от конкретных инструментов и низкий уровень регламентации, что позволяет расширять процесс для адаптации к различным типам проектов, от небольших команд до более сложных инициатив [7]. Например, процесс можно дополнять подключаемыми модулями для поддержки сервис-ориентированной архитектуры, территориально распределенных команд или специфических технологий, таких как J2EE, сохраняя при этом базовую структуру [9]. В плане легкости OpenUP рекомендует практики, типичные для agile-методологий, такие как измерение скорости команды в story points, проведение ежедневных митингов для координации и ретроспектив по итогам каждой итерации, чтобы анализировать успехи и улучшать подход [8]. Кроме того, вводится концепция микрошагов, которая подразумевает разбиение работы на небольшие единицы продолжительностью от нескольких часов до дней, обеспечивая постоянный измеримый прогресс и короткие циклы обратной связи для быстрой адаптации [9]. Это способствует непрерывному созданию работающего продукта, аналогично принципам Extreme Programming, где акцент на baby-steps или малых инкрементах [9].

С другой стороны, OpenUP сохраняет элементы тяжести из RUP, предоставляя готовые

шаблоны для артефактов, таких как описание видения продукта, анализ требований, дизайн, архитектура и тестовая документация, а также контрольные списки для проверки полноты и корректности этих материалов [8]. Формализованный процесс описывает последовательность активностей, цели команды и зоны ответственности ролей, но в упрощенном виде по сравнению с полным RUP, делая его более доступным для команд без экстремальной зрелости [8]. Основные принципы OpenUP включают совместную работу для согласования интересов и достижения общего понимания среди участников, развитие проекта через непрерывную обратную связь и совершенствование, концентрацию на архитектурных вопросах на ранних стадиях для минимизации рисков и организации разработки, а также выравнивание конкурентных преимуществ для максимизации ценности для заинтересованных лиц [7]. Эти принципы отражают прагматичную agile-философию, где коллективный подход к разработке ставится во главу угла, подчеркивая самоорганизацию команды и отказ от директивного управления в пользу тренирующего стиля лидерства [9].

Жизненный цикл проекта в OpenUP разделен на четыре фазы, аналогичные RUP: начальная фаза (inception), где определяются масштаб и задачи, фаза уточнения (elaboration) для согласования архитектуры и снижения рисков, фаза конструирования (construction) для основной реализации и фаза передачи (transition) для финальной отделки и развертывания [7]. Каждая фаза завершается контрольной точкой, где заинтересованные лица оценивают прогресс, отвечая на ключевые вопросы, такие как готовность архитектуры или приемлемость созданной ценности, что позволяет timely принимать решения о продолжении, корректировке или даже отмене проекта [9]. Это обеспечивает прозрачность и контроль, особенно важные для заинтересованных сторон, которые нуждаются в предсказуемости финансирования и рисков [9]. Внутри фаз проект организуется в итерации — планируемые интервалы продолжительностью в недели, где команда фокусируется на поставке работоспособной версии продукта [7]. План итерации определяет ожидаемые результаты, а команда самоорганизуется для распределения задач, начиная с определения элементов работы и переходя к их детализации и выполнению [7].

Жизненный цикл итерации в OpenUP подчеркивает нацеленность на постепенное наращивание ценности через стабильные сборки, которые демонстрируются заинтересованным лицам для получения отзывов [9]. Итерация начинается с планирования, где отбираются высокоприоритетные элементы работы с учетом динамической оценки объема, затем следует основная работа с микрошагами, еженедельными стабильными сборками для поддержания качества и заканчивается оценкой продукта плюс ретроспективой для улучшения процесса [9]. Микрошаги здесь играют центральную роль: они разбивают задачи на управляемые части, такие как идентификация заинтересованных лиц, разработка подпотоков прецедентов или согласование технического подхода, обеспечивая ежедневный прогресс и прозрачность через открытые обсуждения на митингах [9]. Это способствует предотвращению аналитического паралича и фокусирует усилия на создании исполняемого кода, снижая риски за счет раннего выявления проблем [9].

В отношении анализа и проектирования OpenUP продвигает гибкое моделирование (AMDD), где документация создается только по мере необходимости для ближайшего будущего, избегая излишней бюрократии [8]. Тестирование интегрируется рано с использованием checklists

для верификации артефактов, что усиливает дисциплину без перегрузки [8]. Организация взаимодействия в команде строится на принципах самоорганизации, где члены коллектива сами распределяют обязанности, предлагают кандидатуры для задач и видят себя в первую очередь как часть коллектива, а не как отдельные роли [9]. Руководитель проекта выступает как коуч, обеспечивая прозрачность и ответственность через ежедневные обсуждения и инструменты совместной работы [9]. Приоритеты OpenUP акцентируют гибкость и минимализм: процесс минимизирует ненужные детали, фокусируясь на сотрудничестве и адаптации, чтобы максимизировать ценность при минимальных усилиях [9]. В итоге, OpenUP подходит для проектов, где нужна формализация без излишеств, и может быть интегрирован с инструментами вроде Devprom ALM, где предустановленные шаблоны автоматизируют процесс от ролей до структуры требований [8]. Это делает его привлекательным для команд, стремящихся к балансу между дисциплиной и agile-гибкостью, с возможностью эволюции через сообщество Eclipse Process Framework [9].

1.3 SCRUM

Scrum представляет собой легковесный фреймворк, предназначенный для помощи людям, командам и организациям в создании ценности через адаптивные решения для комплексных проблем [10]. Этот подход возник в начале 1990-х годов благодаря усилиям Кена Швабера и Джеффа Сазерленда, которые стремились упростить управление проектами в условиях неопределенности, и с тех пор он эволюционировал, становясь все более популярным в различных областях, включая разработку программного обеспечения, исследования и аналитику [10]. В отличие от традиционных методов, Scrum подчеркивает гибкость и сотрудничество, позволяя командам быстро реагировать на изменения, что делает его одним из ключевых представителей agile-подходов [11]. По сути, Scrum не является строгой методологией с детальными инструкциями, а скорее набором правил и ориентиров, которые направляют взаимодействия людей, помогая оптимизировать процессы на основе реального опыта [10]. Его применение особенно эффективно в проектах, где требования могут меняться, а конечный продукт формируется итеративно, с фокусом на доставке ценности клиенту [2]. Основная идея Scrum заключается в том, чтобы Scrum Master способствовал созданию среды, где Product Owner упорядочивает работу в Product Backlog, Scrum Team превращает ее в ценный инкремент во время Sprint, а затем все заинтересованные стороны инспектируют результаты для корректировки следующего цикла [10]. Такой циклический процесс повторяется, обеспечивая постоянное улучшение и адаптацию к новым условиям [10]. Теоретическая основа Scrum строится на эмпиризме, где знания черпаются из опыта, а решения принимаются на основе наблюдений, а также на бережливом мышлении, которое минимизирует потери и сосредотачивается на самом важном [10]. Это позволяет оптимизировать предсказуемость и управление рисками через итеративный и инкрементальный подход, где группы людей, обладающие необходимыми навыками, делятся знаниями и самоорганизуются для выполнения задач [10]. Центральными элементами эмпиризма в Scrum являются три столпа: прозрачность, инспекция и адаптация, которые обеспечивают видимость процессов и результатов для всех участников, позволяя timely выявлять проблемы и корректировать курс [10]. Прозрачность подразумевает, что все аспекты работы должны быть

видимы как исполнителям, так и получателям, чтобы избежать решений, основанных на неполной информации, и минимизировать риски [10]. Инспекция требует регулярной проверки артефактов и прогресса для обнаружения отклонений, а адаптация обеспечивает быструю корректировку процессов или материалов, если они выходят за допустимые пределы [10]. Эти столпы реализуются через четыре формальных события, объединенных в контейнер Sprint, который является основным циклом работы в Scrum [10]. Успех Scrum во многом зависит от стремления к пяти ценностям: приверженности, сфокусированности, открытости, уважения и смелости, которые определяют поведение и взаимодействия в команде [10]. Приверженность означает, что команда посвящена достижению целей и взаимной поддержке, сфокусированность подразумевает концентрацию на ключевом прогрессе в Sprint, открытость способствует честному обсуждению вызовов, уважение признает профессионализм каждого, а смелость позволяет решать сложные проблемы правильным образом [10]. Эти ценности укрепляют эмпирические столпы, строя доверие между участниками и заинтересованными сторонами [10]. В структуре Scrum ключевыми ролями являются Product Owner, Scrum Master и Developers, образующие Scrum Team, которая является самоуправляемой и кросс-функциональной, способной самостоятельно определять, как достигать целей [10]. Product Owner отвечает за максимизацию ценности продукта, управляя Product Backlog — упорядоченным списком работ, включая требования, улучшения и исправления, который эволюционирует по мере получения обратной связи [10]. Scrum Master выступает как фасилитатор, помогая команде понимать и применять Scrum, удаляя препятствия и способствуя улучшениям, но не управляя командой напрямую [10]. Developers, в широком смысле, включают всех, кто создает инкремент, фокусируясь на доставке работающего продукта в конце каждого Sprint [10]. Scrum организует работу вокруг событий, которые обеспечивают регулярность и минимизируют ненужные встречи: Sprint — фиксированный период (обычно до месяца), где создается потенциально релизный инкремент; Sprint Planning — планирование того, что будет сделано и как; Daily Scrum — ежедневная 15-минутная встреча для синхронизации и адаптации плана; Sprint Review — инспекция инкремента с заинтересованными сторонами для получения обратной связи; Sprint Retrospective — анализ, что прошло хорошо и что улучшить для повышения эффективности [10]. Эти события способствуют прозрачности и адаптации, делая процесс предсказуемым [10]. Артефакты Scrum — Product Backlog, Sprint Backlog и Increment — представляют работу и ценность, с обязательствами (Product Goal, Sprint Goal, Definition of Done), которые усиливают фокус и прозрачность [10]. В сравнении с другими agile-методами, Scrum фокусируется на управлении проектами, повышая вероятность успешной разработки через короткие циклы и вовлеченность пользователей, в отличие от, например, XP, который больше акцентирует на практиках программирования [11]. Его приоритеты включают гибкость к изменениям, клиентскую ценность через частые доставки и сотрудничество, что делает его подходящим для динамичных сред, где традиционные план-driven подходы терпят неудачу из-за неспособности адаптироваться [11]. В практике Scrum подчеркивает спринты как короткие итерации (2-4 недели), с минимальными совещаниями, но высокой частотой, где команда обсуждает прогресс и проблемы, используя журнал спринтов для планирования [2]. Это позволяет быстро запускать проекты с минимальным бюджетом, обеспечивать ежедневный контроль и вносить правки по ходу, хотя требует высокой квалификации команды и может

создавать путаницу при частых изменениях [2]. В итоге, Scrum идеален для команд до десяти человек в проектах с длинным жизненным циклом, требующих адаптации к рынку, и его использование растет, как показывают тренды в IT-гигантах вроде Google и Amazon [2].

1.4 V-Model

V-Model представляет собой одну из классических методологий разработки программного обеспечения, которая возникла как эволюция каскадной модели и подчеркивает важность верификации и валидации на протяжении всего жизненного цикла проекта. Эта модель получила название благодаря своей графической форме, напоминающей букву V, где левая сторона отражает фазы декомпозиции требований и создания спецификаций, а правая сторона фокусируется на интеграции компонентов и их валидации [12]. В отличие от линейных подходов, таких как waterfall, V-Model устанавливает прямые связи между этапами разработки и соответствующими этапами тестирования, что позволяет начинать планирование тестов еще на ранних стадиях, когда формулируются требования, и для каждого уровня разрабатывается отдельный тест-план с определением ожидаемых результатов, критериев входа и выхода [13]. Такая структура обеспечивает, что верификация проводится против технических требований, а валидация ориентирована на соответствие реальным нуждам пользователя, что можно выразить вопросами: "Строим ли мы правильную вещь?" для валидации и "Строим ли мы ее правильно?" для верификации [12]. V-Model делится на три основных типа: немецкий V-Modell, используемый в государственных проектах Германии как эквивалент PRINCE2 с акцентом на доказуемость продуктов левой стороны V для правой стороны; общая модель тестирования, широко применяемая в сообществе тестировщиков как иллюстративное *depiction software development process* по стандартам ISTQB; и стандарт США для системного жизненного цикла, более детальный и строгий, чем европейские аналоги [12]. В контексте разработки ПО V-Model подчеркивает минимизацию рисков через стандартизированные подходы, улучшение качества за счет ранней проверки промежуточных результатов, снижение общих затрат путем прозрачного расчета усилий и повышение коммуникации между заинтересованными сторонами через унифицированные описания элементов [12]. Левая ветвь V включает спецификацию потока, начиная с пользовательских требований, функциональных спецификаций и дизайна, в то время как правая ветвь охватывает тестирование, такое как *installation qualification*, *operational qualification* и *performance qualification*, с возможным развитием через кастомизацию, конфигурацию или кодирование в зависимости от типа системы [12]. Как модифицированная версия waterfall, V-Shaped Model, часто называемая *verification and validation model*, отличается нелинейным потоком, где процессы ветвятся, и развитие зависит от верификации предыдущих стадий, представляя прямые отношения между фазами разработки и тестирования, что повышает шансы на успех по сравнению с чистым waterfall, особенно для небольших проектов [14]. Преимущества этой модели включают простоту использования, четкие результаты на каждом этапе, раннее тестирование для повышения вероятности успеха и *suitability* для маломасштабных программных проектов с фиксированными требованиями [14]. Однако V-Model сохраняет жесткость waterfall, с минимальной гибкостью и высокими затратами на корректировки, отсутствием ранних прототипов, поскольку *software* разрабатывается только на этапе реализации, и неясными путями

разрешения проблем, выявленных во время тестирования [14]. В системах инженерии V-Model способствует улучшению cost-effectiveness сложных систем через раннюю идентификацию целей, концепцию операций, тщательные требования, детальный дизайн, реализацию, rigorous acceptance testing для верификации и измерение эффективности для валидации, с traceability всех элементов дизайна и тестов к требованиям [12]. Применение V-Model особенно актуально в регулируемых отраслях, таких как федеральная администрация Германии, оборонные проекты и коммерческие программы в США, где модель появилась независимо в конце 1980-х годов, включая разработки IABG для министерства обороны и стандарты DoD для систем с hardware, software и human interaction [12]. Для проектов, требующих тщательного тестирования, наличия квалифицированных инженеров, особенно тестеров, и фиксированных требований в малых или средних масштабах, V-Model оправдывает свой фокус на validation and verification, с контролем на каждом этапе для перехода к следующему и отдельными уровнями тестового покрытия [13]. В отличие от итеративных моделей, V-Model не предполагает возвратов во времени, подчеркивая движение от левой к правой стороне с итерациями только по вертикали в иерархии системы, что делает ее фундаментальной чертой для управления проектами с акцентом на качество и раннее выявление отклонений [12]. В образовательных материалах по проектированию ПО подчеркивается, что V-Model улучшает классическую каскадную модель за счет встроенного контроля процессов и стратегий тестирования, определяемых параллельно с разработкой, что позволяет прогнозировать результаты и устанавливать критерии завершения этапов [13]. Сравнивая с другими методами, такими как parallel или iterative, V-Shaped Model выделяется своей способностью к раннему тестированию, но уступает в гибкости для сложных проектов с изменяющимися требованиями, где параллельные субпроекты или итеративные циклы могут сократить время доставки [14]. В целом, приоритеты V-Model сосредоточены на качестве через раннее тестирование, строгой верификации и валидации, делая ее подходящей для сред, где риски должны минимизироваться через стандартизацию, хотя она не охватывает аспекты, такие как контракты на услуги, полное обслуживание системы или организационные рамки за пределами проекта, требуя адаптаций для конкретных нужд [12].

1.5 RAD (Rapid Application Development)

Методология быстрой разработки приложений, известная как RAD, представляет собой подход, ориентированный на ускорение процесса создания программного обеспечения за счет итеративного и инкрементного стиля работы, где особое внимание уделяется активному участию пользователя или заказчика на протяжении всего цикла разработки. Эта методология возникла как ответ на ограничения традиционных последовательных моделей, таких как водопадная, где жесткие этапы часто приводят к задержкам и несоответствию конечного продукта актуальным нуждам. В RAD основной акцент делается на достижении готового проекта в сжатые сроки при фиксированном бюджете, с учетом возможных изменений требований в процессе проектирования и реализации, что делает ее особенно подходящей для динамичных сред, где скорость вывода продукта на рынок играет решающую роль [5]. Концепция RAD подразумевает использование готовых решений, библиотек и инструментов для минимизации оригинальной разработки сложных алгоритмов, что позволяет сосредоточиться на интеграции компонентов

и быстром прототипировании, аналогично тому, как из полуфабрикатов быстро собирается готовый продукт, без лишних усилий на создание всего с нуля [4].

Принципы RAD строятся вокруг итеративности, где разработка происходит в коротких циклах, каждый из которых включает сбор требований, проектирование, реализацию и тестирование, с постоянным получением обратной связи от пользователя, что обеспечивает гибкость и адаптивность к изменениям. Вовлеченность заказчика начинается с ранних этапов и продолжается до финальной сдачи, что помогает избежать недоразумений и доработок на поздних стадиях, снижая общие затраты. Кроме того, методология подчеркивает клиентоориентированность, простоту реализации и использование мощных инструментов для автоматизации рутинных задач, таких как генерация кода или визуальное моделирование интерфейсов. Однако RAD не универсальна и имеет ограничения, такие как сжатые сроки проекта, обычно не превышающие 60-90 дней, и необходимость декомпозиции крупных систем на модульные компоненты, чтобы избежать хаоса в больших командах [5]. В сравнении с итеративными моделями, RAD усиливает фокус на скорости, жертвуя иногда глубиной архитектуры в пользу быстрого развертывания, что делает ее похожей на мини-водопады в рамках каждого цикла, но с акцентом на пользовательский фидбек для корректировки [14].

Фазы в RAD обычно делятся на четыре основных этапа: планирование требований, где определяются цели и приоритеты с участием пользователя; дизайн, включающий создание прототипов и моделирование интерфейса; строительство, где происходит кодирование и интеграция компонентов; и внедрение, с финальным тестированием и передачей системы в эксплуатацию. На этапе планирования акцент на быстром сборе и уточнении требований через совместные сессии с заказчиком, что позволяет сразу выявить ключевые функции и избежать перегрузки ненужными деталями. Дизайн в RAD часто использует инструменты для быстрого прототипирования, такие как визуальные редакторы, чтобы пользователь мог увидеть и оценить интерфейс на ранней стадии, корректируя его до начала полноценной реализации. Строительство подразумевает итеративное добавление функциональности, с использованием готовых библиотек для ускорения, а внедрение включает не только развертывание, но и обучение пользователей, с возможностью быстрой итерации на основе первых отзывов [4]. В отличие от более структурированных подходов, фазы в RAD перетекают друг в друга без жестких границ, что способствует адаптивности, но требует высокой квалификации команды для управления рисками, такими как неполная интеграция модулей или игнорирование долгосрочной масштабируемости [5].

Приоритеты RAD сосредоточены на скорости разработки, низкой стоимости и высоком качестве за счет минимизации бюрократии и максимального использования существующих решений, что делает ее идеальной для проектов с изменяющимися требованиями и необходимостью быстрого прототипирования. Скорость достигается через короткие итерации и фокус на минимально жизнеспособном продукте, где вместо глубокого анализа рисков или сложной архитектуры предпочтение отдается практической реализации и пользовательскому фидбеку для быстрой корректировки. Адаптивность проявляется в способности вносить изменения на любом этапе без значительных потерь времени, в отличие от последовательных моделей, где возвраты к предыдущим фазам обходятся дорого. Однако это подразумевает риски для крупных

проектов, где требуется большой коллектив разработчиков и тщательная декомпозиция, иначе методология может привести к невыполнению сроков или снижению качества из-за спешки [5]. В контексте сравнения с итеративными методами, RAD подчеркивает продуктивность над инженерной глубиной, избегая сложных практик вроде парного программирования, но интегрируя элементы быстрого добавления фич, видимых пользователю, что делает ее подходящей для стартапов или проектов поддержки, где энтузиазм команды и простота процесса важнее строгой формализации [4]. В целом, RAD эффективна в ситуациях, когда проект не слишком сложен, риски минимальны, а основной вызов — это время на рынок, позволяя создавать работающие приложения быстрее, чем в традиционных моделях, но с учетом того, что для исследовательских или высокорисковых задач она может оказаться недостаточно крепкой [14].

2 Сравнительная характеристика методологий

2.1 Приоритеты в каждом подходе

Каждый подход к разработке программного обеспечения устанавливает свои уникальные приоритеты, которые определяют фокус на определенных аспектах проекта, таких как управление рисками, обеспечение качества или быстрая адаптация к изменениям, и эти приоритеты напрямую влияют на то, как методология справляется с динамикой современных бизнес-требований [11]. В рамках сравнительного анализа важно отметить, что традиционные методологии, такие как V-Model, подчеркивают предсказуемость и тщательную документацию, в то время как agile-подходы, включая SCRUM и OpenUP, ориентированы на гибкость и ценность для клиента, что позволяет лучше справляться с неопределенностью в требованиях [11]. Оценка "goodness" методологий, предложенная в OPP-фреймворке, подразумевает анализ их адекватности, способности организации к реализации и эффективности через призму целей, принципов и практик, где приоритеты играют ключевую роль в достижении желаемых результатов [15]. Многокритериальная классификация методологий, учитывающая такие факторы, как стабильность требований, надежность и функциональность, помогает выделить, как эти приоритеты адаптируются к разным уровням сложности проектов, от простых до высоконадежных систем [3].

В Rational Unified Process (RUP) основной приоритет отдается управлению рисками и архитектуре системы, что делает этот подход особенно подходящим для крупных проектов, где необходимо минимизировать неопределенности на ранних этапах через итеративное планирование и фокус на ключевых артефактах [3]. Здесь акцент на архитектуре подразумевает, что разработка строится вокруг стабильного каркаса, который позволяет интегрировать изменения без разрушения общей структуры, а управление рисками реализуется через регулярные оценки и корректировки в каждой фазе, обеспечивая баланс между предсказуемостью и адаптивностью [11]. Такой приоритет помогает избежать перерасхода ресурсов в проектах с высокой сложностью, где, по OPP-фреймворку, цели вроде гибкости достигаются через принципы, такие как аккомодация изменений, и практики, включая итеративное прототипирование [15]. В контексте многокритериальной классификации RUP относится к процессно-ориентированным методологиям для надежных систем со средней сложностью, где приоритет архитектуры обеспечивает долгосрочную поддерживаемость, но может замедлить начальные этапы в сравнении с более легковесными подходами [3]. Это особенно заметно в проектах, где требования частично определены, и приоритет на рисках позволяет предвидеть потенциальные сбои, делая RUP предпочтительным для enterprise-уровня [11].

OpenUP, как облегченная версия RUP, ставит во главу угла минимализм и сотрудничество, фокусируясь на создании ценности через простые практики и активное вовлечение команды, что делает его идеальным для небольших или средних проектов с изменяющимися требованиями [3]. Минимализм здесь проявляется в отказе от избыточной документации в пользу работающего продукта, а приоритет на сотрудничестве подразумевает самоорганизующиеся команды, где коллективные решения ускоряют итерации и повышают мотивацию [11]. Согласно OPP-фреймворку, такие приоритеты поддерживают цели agile-философии, как гибкость и эф-

фективность, через принципы вроде аккомодации изменений и практики, ориентированные на командную работу, что позволяет оценивать "goodness" методологии по ее способности к быстрой адаптации без потери качества [15]. В многокритериальной классификации OpenUP классифицируется как инструментальная методология для систем средней надежности и простоты, где приоритет минимализма снижает overhead, но требует дисциплины в команде для избежания хаоса в крупных проектах [3]. Этот подход особенно выигрывает в сравнении с традиционными, подчеркивая ценность для клиента через короткие циклы, что aligns с agile-манifestом и делает его гибридным между структурированным RUP и чисто agile-методами [11].

SCRUM акцентирует внимание на гибкости и клиентской ценности, где приоритет отдается быстрой доставке инкрементов продукта, позволяя адаптироваться к изменениям в реальном времени через спринты и ретроспективы [11]. Гибкость здесь достигается за счет эмпирического подхода, основанного на прозрачности, инспекции и адаптации, что делает методологию особенно эффективной в динамичных средах, где требования эволюционируют [3]. По OPP-фреймворку, такие приоритеты соответствуют agile-целям, как удовлетворение клиента через непрерывную доставку, и поддерживаются принципами вроде приветствия изменений и практиками, включая ежедневные стендапы, которые усиливают командную синергию [15]. В сравнении с традиционными подходами SCRUM выигрывает в скорости реакции, но может уступать в проектах с высокой надежностью, где нужна строгая документация [11]. Многокритериальная классификация размещает SCRUM в категории командных методологий для гибких требований и средней надежности, подчеркивая его приоритет на ценности как способе минимизировать отходы и максимизировать бизнес-выгоду, что особенно актуально для софта с частыми обновлениями [3].

V-Model в первую очередь приоритизирует качество и верификацию, структурируя процесс в форме V, где левая сторона фокусируется на декомпозиции требований, а правая — на интеграции и тестировании, обеспечивая раннее выявление дефектов [3]. Этот приоритет на качестве подразумевает, что каждый этап разработки напрямую связан с соответствующим этапом тестирования, что минимизирует риски в проектах с жесткими требованиями к надежности [11]. В OPP-фреймворке такая ориентация поддерживает цели вроде надежности через принципы тщательной валидации и практики, включая параллельное планирование тестов, что позволяет оценивать эффективность методологии по ее способности предотвращать ошибки на ранних стадиях [15]. По сравнению с agile-подходами V-Model менее гибок к изменениям, но превосходит в предсказуемости для критически важных систем, таких как в авиации или медицине [11]. В многокритериальной классификации он относится к жестким методологиям для высоконадежных и сложных систем, где приоритет верификации обеспечивает соответствие стандартам, но может увеличить время на начальные фазы в динамичных проектах [3].

Наконец, RAD (Rapid Application Development) уделяет первостепенное внимание скорости и пользовательскому фидбеку, используя итеративное прототипирование для быстрой итерации идей и корректировок на основе отзывов клиента [3]. Скорость здесь достигается через минимальные циклы разработки, где приоритет на фидбеке позволяет быстро адаптировать продукт к реальным нуждам, минимизируя время от концепции до развертывания [11]. Согласно OPP-фреймворку, эти приоритеты align с agile-целями, такими как быстрая доставка ценно-

сти, через принципы вовлеченности пользователей и практики, вроде совместного дизайна, что повышает "goodness" в проектах с нестабильными требованиями [15]. В сравнении с структурированными подходами, как RUP или V-Model, RAD выигрывает в адаптивности, но может страдать от недостатка глубины в крупных системах [11]. Многокритериальная классификация позиционирует RAD как инструментальную методологию для гибких требований и средней сложности, где приоритет скорости делает ее подходящей для стартапов или MVP, но требует осторожности в высоконадежных контекстах [3].

2.2 Политики в отношении анализа и проектирования

В области анализа и проектирования различные методологии разработки программного обеспечения предлагают свои уникальные подходы, которые в значительной степени определяются общими принципами каждой из них, такими как степень структурированности, вовлеченность заинтересованных сторон и акцент на итеративности или последовательности. Эти политики не только влияют на то, как собираются и уточняются требования, но и на способ создания моделей системы, что в конечном итоге сказывается на общей эффективности проекта. Например, в Rational Unified Process анализ и проектирование тесно интегрированы в начальные фазы, где особое внимание уделяется управлению требованиями и архитектурным решениям, чтобы минимизировать риски на ранних этапах. В inception-фазе RUP происходит предварительный анализ предметной области, формулировка видения проекта и выявление ключевых требований, что позволяет оценить feasibility и установить базовые цели, а в elaboration-фазе углубляется проектирование с фокусом на детальные спецификации, включая моделирование сущностей и взаимодействий, часто с использованием UML-диаграмм для описания архитектуры системы [6]. Такой подход подчеркивает итеративный характер, где анализ не является разовым действием, а повторяется в каждой итерации, чтобы адаптировать требования к изменяющимся условиям, и при этом большое значение придается формализации задач через технические задания и спецификации, что особенно актуально для сложных систем, таких как те, что разрабатываются для платформ вроде System z, где нужно учитывать интеграцию с существующими компонентами. В отличие от этого, OpenUP, как облегченная версия RUP, ориентирована на более гибкое моделирование, где анализ и проектирование осуществляются через микрошаги и итеративные циклы, с акцентом на сотрудничество и минимализм, чтобы избежать излишней бюрократии. В OpenUP анализ начинается с согласования интересов заинтересованных сторон и создания общего понимания через совместные обсуждения, а проектирование фокусируется на архитектурных вопросах на ранних стадиях для минимизации рисков, используя прагматичные инструменты вроде визуализации процесса поставки, где задачи по анализу требований интегрируются в жизненный цикл итерации, включая планирование и уточнение элементов работы в виде backlog-подобных списков [9]. Это позволяет быстро адаптировать дизайн к обратной связи, делая процесс менее жестким, чем в полном RUP, и подчеркивая развитие через непрерывную обратную связь, где моделирование не всегда требует детальных диаграмм, а может ограничиваться простыми эскизами или прототипами для подтверждения концепций. Переходя к Scrum, здесь политика в отношении анализа и проектирования сильно ориентирована на эмпиризм и адаптивность, где анализ осуществляется через управление Product Backlog, который

представляет собой динамичный список требований, приоритизированных Product Owner'ом на основе ценности для клиента, и проектирование emerges в ходе спринтов без жестких предварительных фаз. В Scrum анализ не выделен в отдельные этапы, а интегрирован в ежедневные взаимодействия, такие как Sprint Planning, где команда разбирает backlog items в задачи, и Retrospective, где адаптируются процессы; это подразумевает, что проектирование фокусируется на создании инкрементов ценности, с прозрачностью через Definition of Done, где требования уточняются итеративно на основе инспекции и адаптации, без глубокого предварительного моделирования, но с сильным акцентом на командное сотрудничество для быстрого разрешения неопределенностей [10]. Такой подход контрастирует с более структурированными методологиями, поскольку здесь анализ требований происходит в реальном времени, часто через user stories или аналогичные артефакты, что делает его подходящим для проектов с изменчивыми требованиями, где фокус на гибкости перевешивает детальное планирование. В V-Model политика анализа и проектирования следует строгой последовательной логике, где левая сторона "V" посвящена декомпозиции требований от user needs к детальным спецификациям, включая functional и design specifications, с акцентом на верификацию на каждом уровне, чтобы обеспечить traceability от требований к конечному продукту. Анализ в V-Model начинается с user requirement specifications, переходя к functional и design, где моделирование осуществляется через графические представления, такие как entity-relationship diagrams или data flow diagrams, и проектирование подразумевает создание testable specifications, которые напрямую связаны с правой стороной для валидации; это минимизирует риски через раннюю проверку, но ограничивает гибкость, поскольку изменения на поздних этапах требуют возврата к началу, делая подход идеальным для проектов с фиксированными требованиями, где качество и compliance с регуляциями приоритетны [12]. Наконец, в Rapid Application Development анализ и проектирование построены на итеративном вовлечении пользователя, где анализ фокусируется на быстром сборе требований через прототипирование, а проектирование осуществляется в коротких циклах с акцентом на адаптивность и клиентоориентированность, без глубоких предварительных моделей, но с использованием готовых библиотек и инструментов для ускорения. В RAD фазы анализа включают планирование с участием заказчика для динамического формирования требований, а проектирование ориентировано на инкрементные улучшения, где изменения вносятся на любом этапе, что снижает затраты на доработки, но требует высокой квалификации команды и может привести к рискам в крупных проектах из-за отсутствия строгой структуры; при этом приоритеты смещены к скорости и пользовательскому фидбеку, делая анализ менее формализованным, чем в V-Model или RUP [5]. Сравнивая эти политики, можно заметить, что RUP и OpenUP предлагают баланс между структурой и гибкостью, с сильным фокусом на моделях и требованиях в ранних фазах, в то время как Scrum и RAD подчеркивают адаптивность и минимальный анализ для быстрого прогресса, в отличие от V-Model, где акцент на тщательной декомпозиции и traceability для обеспечения качества, что делает выбор зависящим от характера проекта: для сложных систем с рисками подойдет RUP с его elaboration-фазой [6], для динамичных команд — Scrum с backlog-управлением [10], а для быстрого прототипирования — RAD с итеративным анализом [5]. В OpenUP гибкое моделирование позволяет интегрировать элементы из других подходов, делая его переходным между структурированным RUP и

agile-подобным Scrum [9], тогда как V-Model остается наиболее жестким, требуя полной спецификации перед переходом к реализации, что может замедлить процесс в изменчивой среде [12]. Таким образом, политики анализа и проектирования в этих методологиях отражают их общую философию: от предсказуемости и контроля в V-Model к эмпирической адаптации в Scrum, с промежуточными вариантами в RUP, OpenUP и RAD, где гибкость требований коррелирует с уровнем вовлеченности команды и заказчика.

2.3 Политики в отношении написания и организации кода

В рамках сравнительной характеристики методологий разработки программного обеспечения особое внимание стоит уделить политикам в отношении написания и организации кода, поскольку этот аспект напрямую влияет на эффективность команды, качество продукта и способность адаптироваться к изменениям в проекте. Каждая методология предлагает свой подход к этим процессам, балансируя между структурированностью, скоростью и коллективной ответственностью, что позволяет разработчикам выбирать оптимальный вариант в зависимости от масштаба проекта, квалификации команды и бизнес-требований. В Rational Unified Process, например, написание кода осуществляется в итеративном режиме, где каждый цикл разработки включает создание инкрементов программного обеспечения, что позволяет постепенно наращивать функциональность на основе ранее проверенной архитектуры, минимизируя риски переработки [6]. Это подразумевает, что организация кода в RUP ориентирована на фазовую структуру, где в фазе *elaboration* происходит детальное проектирование, а в *construction* – основная реализация, с акцентом на интеграцию компонентов и соблюдение стандартов, таких как использование шаблонов для артефактов, что обеспечивает последовательность и *traceability* от требований к коду [6]. Такой подход способствует тому, что код не пишется в изоляции, а организуется вокруг ключевых сценариев использования, с регулярными ревью для выявления несоответствий архитектуре, что особенно полезно в крупных проектах, где задействованы несколько команд [6]. В отличие от этого, OpenUP, как облегченная версия унифицированного процесса, вводит политику микро-шагов в написании кода, где разработка ведется через небольшие, непрерывные инкременты, повторяющие принцип создания работающего продукта на каждой итерации, что позволяет быстро реагировать на изменения без масштабной перестройки [8]. Организация кода здесь подчеркивает коллективный подход, с использованием готовых шаблонов для дизайна и кода, а также *checklists* для проверки полноты, что делает процесс менее регламентированным, но более ориентированным на зрелость команды, где разработчики делят ответственность за код через ежедневные митинги и ретроспективы [8]. Это способствует повышению качества кода за счет фокуса на минимально необходимой документации и гибком моделировании, где код организуется не по строгим фазам, а по приоритетам бэклога, что идеально для средних проектов с акцентом на сотрудничество [8]. Переходя к Scrum, здесь политика написания кода строится вокруг спринтов – фиксированных периодов, в течение которых команда реализует выбранные задачи из *product backlog*, с ежедневными митингами (*Daily Scrum*), где обсуждается прогресс, предстоящие задачи и препятствия, что обеспечивает высокую степень организации и координации внутри команды [10]. В Scrum код пишется инкрементально, с акцентом на создание потенциально готового к релизу инкремента

в конце каждого спринта, что подразумевает коллективную ответственность за качество кода, без жестких стандартов, но с ценностями открытости и уважения, где разработчики самоорганизуются для распределения задач [10]. Это отличает Scrum от более структурированных подходов, поскольку организация кода здесь опирается на эмпиризм – прозрачность прогресса через инспекцию и адаптацию, что позволяет быстро корректировать код на основе отзывов, но требует высокой дисциплины команды для избежания хаоса в крупных проектах [10]. В V-Model политика написания кода интегрирована в линейную, но ветвящуюся структуру, где левая сторона V фокусируется на декомпозиции требований и дизайне, а код реализуется после детального проектирования, с последующей верификацией на правой стороне для обеспечения соответствия спецификациям [13]. Организация кода в этой модели подчеркивает стандарты верификации, где каждый модуль кода проверяется на соответствие дизайну через unit-тесты и интеграцию, что делает процесс предсказуемым и ориентированным на качество, но менее гибким для изменений, поскольку переработка кода требует возврата к предыдущим этапам [12]. Такой подход особенно эффективен в проектах с фиксированными требованиями, где организация кода строится на четких критериях входа и выхода для каждого уровня, минимизируя ошибки за счет ранней подготовки тестовых планов параллельно с написанием кода [14]. Наконец, в Rapid Application Development политика написания кода ориентирована на скорость и итеративность, где разработка ведется через быстрые прототипы с использованием языков и инструментов быстрой разработки, таких как высокоуровневые фреймворки, что позволяет создавать код в короткие сроки с вовлечением пользователей для немедленной обратной связи [13]. Организация кода в RAD подразумевает модульный подход, где каждый инкремент строится независимо, но интегрируется с предыдущими, с акцентом на минимизацию изменений между сборками для избежания проблем с совместимостью, что делает процесс подходящим для проектов с неопределенными требованиями, но требует квалифицированных специалистов для управления рисками интеграции [14]. Сравнивая эти политики, можно отметить, что итеративность в написании кода является общей чертой для RUP, OpenUP, Scrum и RAD, позволяя постепенно улучшать код на основе обратной связи, в то время как V-Model предпочитает последовательный подход с сильным акцентом на верификацию, что снижает риски в регулируемых отраслях, но может замедлить разработку [14]. В плане коллективности Scrum и OpenUP выделяются ежедневными митингами и самоорганизацией, способствующими обмену знаниями и совместной отладке кода, тогда как RUP и V-Model полагаются на формализованные роли и checklists для обеспечения стандартов, что полезно в распределенных командах [8]. Стандарты организации кода в RUP и OpenUP включают шаблоны и checklists для единообразия, в Scrum – ценности для культурной согласованности, в V-Model – строгие критерии верификации, а в RAD – фокус на инструментах для ускорения, но с потенциальными рисками в поддержке долгосрочного кода из-за скорости [6]. В итоге, выбор политики зависит от баланса между скоростью (RAD, Scrum), качеством (V-Model, RUP) и сотрудничеством (OpenUP, Scrum), что позволяет адаптировать подход к конкретным условиям проекта для достижения оптимальных результатов в разработке.

2.4 Политики в отношении тестирования

Политики в отношении тестирования в рассматриваемых методологиях разработки программного обеспечения отражают их фундаментальные подходы к управлению качеством и рисками, а также степень формализации процесса. В RUP тестирование рассматривается как неотъемлемая часть итерационного цикла и одной из ключевых дисциплин на протяжении всего жизненного цикла проекта. Оно начинается уже на ранних фазах, в частности в *elaboration*, где проводится тестирование архитектуры и критически важных сценариев использования для подтверждения стабильности базовой линии системы [6]. По мере продвижения в *construction* фаза акцентирует внимание на систематическом тестировании инкрементов, включая модульное, интеграционное, системное и нагрузочное тестирование, при этом особое значение придается раннему выявлению дефектов, поскольку стоимость их исправления растет экспоненциально на поздних стадиях [6]. В *transition* фаза фокусируется на приемочном тестировании и тестировании в условиях реальной эксплуатации, что позволяет гарантировать готовность продукта к передаче заказчику. Такой подход подчеркивает принцип управления рисками через непрерывную верификацию и валидацию на каждой итерации, что особенно заметно в RUP for System z, где приведены конкретные примеры тестовых сценариев и матриц покрытия для COBOL-приложений и EGL-компонентов [6].

OpenUP наследует многие черты RUP, однако делает акцент на прагматичности и минимизации избыточной документации, что напрямую влияет на политику тестирования. Тестирование в OpenUP встроено в микрошаги итераций и ориентировано на создание полностью протестированных сборок в конце каждой итерации, что является одним из центральных требований процесса [9]. В отличие от более тяжелого RUP здесь явно подчеркивается необходимость стабильных инкрементов, которые проходят автоматизированное и ручное тестирование в рамках одного цикла обратной связи, длительностью от нескольких часов до нескольких дней [9]. OpenUP требует, чтобы каждая итерация завершалась демонстрацией работающего кода, прошедшего комплексное тестирование, включая функциональное, регрессионное и приемочное, при этом *checklists* и простые критерии завершенности (*Definition of Done*) играют роль формального контроля качества [9]. Такой подход позволяет сохранять высокую скорость разработки без потери качества, но при этом требует дисциплины от команды в соблюдении микрошагов, связанных с тестированием, что делает процесс более легким по сравнению с RUP, но все еще структурированным [9].

В Scrum политика тестирования строится вокруг принципа, что каждый инкремент, создаваемый в спринте, должен быть потенциально поставляемым и полностью готовым к использованию, что подразумевает обязательное прохождение всех необходимых видов тестирования внутри спринта [10]. Тестирование не выделяется в отдельную фазу или роль, а является сквозной практикой, выполняемой разработчиками в рамках ежедневных стендапов и спринт-ревью. *Definition of Done*, согласованный командой, включает критерии качества, среди которых обязательно присутствуют успешное модульное тестирование, интеграционное тестирование, автоматизированные тесты и ручное приемочное тестирование, если это необходимо [10]. Такой подход обеспечивает непрерывную интеграцию и тестирование (CI/CD), минимизируя накопле-

ние технического долга и позволяя быстро реагировать на изменения требований [10]. Scrum не предписывает конкретных техник тестирования, оставляя команде свободу выбора инструментов и методов, но жестко требует, чтобы качество проверялось в каждом спринте, что делает тестирование органичной частью разработки, а не отдельным этапом [10].

V-Model представляет собой наиболее формализованную и строгую политику тестирования среди рассматриваемых подходов. Структура V-образной модели напрямую связывает каждый уровень спецификаций на левой стороне (разработка) с соответствующим уровнем тестирования на правой стороне (верификация и валидация), что гарантирует трассируемость и полноту покрытия [12]. На уровне unit testing проверяется соответствие кода низкоуровневым спецификациям, integration testing — взаимодействие компонентов, system testing — соответствие всей системы функциональным и нефункциональным требованиям, а acceptance testing — соответствие ожиданиям пользователей и бизнес-требованиям [12]. Такой подход обеспечивает, что тестирование начинается не после завершения разработки, а параллельно ей: каждый артефакт разработки имеет соответствующий тест-кейс или тестовую стратегию, разработанную еще на этапе спецификации [12]. В результате V-Model минимизирует риски пропуска дефектов на ранних стадиях и особенно эффективна в проектах с высокими требованиями к надежности, таких как системы критического применения, где цена ошибки крайне высока [12].

В RAD тестирование носит итеративный и пользовательско-ориентированный характер, подчиненный главной цели — максимально быстрому созданию работающего прототипа и его доработке [5]. Тестирование проводится в каждом коротком цикле разработки: после создания прототипа следует его демонстрация заказчику, сбор обратной связи и оперативное внесение изменений, при этом тестирование часто выполняется самими пользователями в форме приемочного тестирования на ранних этапах [5]. Такой подход позволяет быстро выявлять несоответствия требованиям, но не предусматривает глубокого систематического тестирования на уровне модулей или интеграции, что может привести к накоплению скрытых дефектов, если проект не декомпозирован должным образом [5]. RAD полагается на высокую вовлеченность заказчика и частые итерации, поэтому тестирование здесь тесно связано с пользовательским фидбеком, а не с формальными тест-планами, что делает его наиболее гибким, но и наименее предсказуемым с точки зрения полноты покрытия [5].

При сравнении становится очевидным, что V-Model предлагает наиболее детализированную и строгую политику тестирования, где каждый шаг разработки имеет зеркальный шаг верификации, что обеспечивает максимальную уверенность в качестве, но требует значительных временных и ресурсных затрат [12]. RUP и OpenUP занимают промежуточное положение, сочетая итеративность с ранним и непрерывным тестированием, при этом RUP более формален и документирован, а OpenUP — прагматичен и ориентирован на минимально необходимые практики [6] [9]. Scrum делает тестирование полностью интегрированным в разработку, превращая его в ежедневную ответственность команды и обеспечивая высокую скорость обратной связи, но без жесткой предписанной структуры уровней тестирования [10]. RAD идет дальше в сторону скорости, полагаясь на пользовательское тестирование и частые демонстрации, что позволяет оперативно корректировать продукт, но оставляет меньше гарантий систематического покрытия дефектов [5]. Таким образом, выбор политики тестирования определяется балансом

между необходимой степенью формализации, скоростью разработки и уровнем допустимого риска в конкретном проекте.

2.5 Политики в отношении внедрения

Внедрение в методологиях разработки программного обеспечения представляет собой критический этап, на котором результаты проекта передаются конечным пользователям или заказчику, обеспечивая плавный переход от разработки к эксплуатации. В RUP этот процесс организован через *dedicated* фазу *transition*, где акцент делается на подготовке к релизу, включая *beta*-тестирование, обучение пользователей и корректировку на основе отзывов, что позволяет минимизировать риски и обеспечить полную готовность системы к производственной среде [6]. Здесь внедрение не является разовым событием, а представляет собой итеративный процесс, интегрированный с предыдущими фазами, где команда фокусируется на развертывании инкрементов, мониторинге производительности и планировании поддержки, подчеркивая важность управления изменениями для долгосрочной устойчивости продукта. В отличие от этого, OpenUP, как упрощенная версия RUP, подчеркивает прагматичный подход к передаче продукта, где внедрение происходит в конце фазы *transition* с минимальными формальностями, ориентированными на быструю интеграцию с существующими процессами заказчика и акцентом на документацию только необходимого уровня, что делает его более гибким для небольших команд [8]. Это позволяет избежать излишней бюрократии, сосредоточившись на практических аспектах, таких как *handover* инструкций и начальная поддержка, чтобы обеспечить *seamless* интеграцию без перегрузки ресурсами. В SCRUM внедрение реализуется через концепцию потенциально *shippable* инкрементов в конце каждого спринта, что подразумевает регулярные релизы работающего продукта, позволяющие заказчику получать ценность *incrementally* и адаптировать систему на основе реального использования, без отдельной фазы, но с сильным акцентом на ретроспективы для улучшения будущих внедрений [10]. Такой подход способствует быстрому отклику на изменения, где команда и *product owner* совместно определяют готовность инкремента к релизу, интегрируя внедрение в повседневный цикл, что особенно полезно в динамичных проектах с частыми корректировками. V-Model, в свою очередь, структурирует внедрение как часть правой стороны V-образной модели, где интеграция компонентов и валидация происходят последовательно после верификации, фокусируясь на строгом соответствии требованиям через уровни тестирования от *unit* до *system acceptance*, что гарантирует высокую надежность перед финальным развертыванием [12]. Здесь политика подчеркивает формализованный процесс, включая документацию для *maintenance* и *repair*, но без итеративных корректировок, что делает его подходящим для проектов с фиксированными требованиями, где внедрение включает тщательную проверку на соответствие *user needs* и минимизацию рисков через *standardized* подходы. Наконец, в RAD внедрение ориентировано на скорость и вовлеченность пользователей, где быстрая сдача прототипов позволяет проводить итеративные релизы в сжатые сроки, часто в пределах 60-90 дней, с акцентом на *immediate feedback* и корректировки на месте, что ускоряет переход к *production* без длительных фаз подготовки [4]. Этот метод подразумевает минимальную документацию для поддержки, полагаясь на *agile-like* циклы, где внедрение интегрируется с дизайном и строительством для достижения *rapid time-to-market*. Сравнивая

эти политики, можно отметить, что RUP и OpenUP предлагают наиболее структурированные подходы к внедрению с dedicated фазами, идеальными для крупных проектов, где необходимы детальные планы поддержки и управления изменениями, в то время как SCRUM и RAD фокусируются на гибкости и частых релизах, что снижает время на внедрение, но требует высокой вовлеченности команды для обработки пост-релизных корректировок [6] [8] [10] [4]. V-Model выделяется своим emphasis на verification-driven интеграции, что обеспечивает качество, но может замедлить процесс в изменяющихся условиях, в отличие от более adaptive методологий вроде SCRUM, где внедрение становится непрерывным потоком ценности [12] [10]. В отношении поддержки после внедрения RUP и V-Model предоставляют формальные механизмы, такие как planning for operation и maintenance, что критично для mission-critical систем, тогда как OpenUP и RAD полагаются на минималистичные политики, предполагая, что поддержка будет эволюционировать на основе user input, а SCRUM интегрирует ее в backlog для будущих спринтов [6] [12] [8] [4] [10]. В итоге, выбор политики внедрения зависит от масштаба проекта: для стабильных требований V-Model обеспечивает надежность, для динамичных - SCRUM и RAD ускоряют адаптацию, а RUP с OpenUP балансируют между структурой и гибкостью, минимизируя риски на этапе перехода к эксплуатации.

2.6 Организация взаимодействия внутри команды (или между командами)

В организации взаимодействия внутри команды и между командами методологии разработки программного обеспечения демонстрируют значительные различия, обусловленные их фундаментальными принципами и подходами к управлению процессами, где акцент может быть сделан на иерархических ролях, самоорганизации или визуализации потока работ для повышения эффективности коллективной деятельности. В Rational Unified Process, или RUP, взаимодействие строится вокруг четко определенных ролей, таких как менеджер проекта, архитектор, дизайнер, программист и тестер, что обеспечивает структурированный подход к распределению обязанностей на протяжении всего жизненного цикла, включая фазы inception, elaboration, construction и transition, где команды координируют усилия через итеративные обзоры и milestones, способствуя интеграции усилий нескольких команд в крупных проектах путем фокуса на общих артефактах, таких как use cases и архитектурные модели [6]. Такой подход подчеркивает формализованное лидерство и ответственность, где менеджеры и архитекторы играют ключевую роль в координации, а взаимодействие между командами достигается через общие процессы и инструменты, такие как Rational Method Composer, что позволяет адаптировать метод для enterprise-уровня, но может создавать барьеры для спонтанного сотрудничества в малых группах. В отличие от этого, Open Unified Process, или OpenUP, как облегченная версия RUP, акцентирует самоорганизацию команд, где участники, включая заинтересованных лиц, сотрудничают в микро-шагах и итерациях, ориентированных на коллективное принятие решений, с ролями, такими как stakeholder и developer, которые способствуют горизонтальному взаимодействию без строгой иерархии, а ежедневные координационные встречи и ретроспективы усиливают обратную связь внутри команды, делая процесс более прагматичным и подходящим для небольших групп, где расширение на несколько команд происходит через общие принципы, такие как баланс между риском и ценностью [9]. Это способствует гибкому обмену знаниями,

но требует зрелости команды для эффективной самоорганизации, в то время как интеграция с другими методологиями, например, через Eclipse Process Framework, позволяет масштабировать взаимодействие на межкомандный уровень без потери фокуса на минимализме. Scrum, в свою очередь, полностью ориентирован на эмпирический подход к взаимодействию, где ключевые роли — Product Owner, отвечающий за приоритизацию backlog, Scrum Master, facilitating процессы, и Development Team, самоорганизующаяся для выполнения спринтов, — обеспечивают прозрачность через ежедневные stand-up встречи, sprint planning, reviews и retrospectives, что усиливает адаптацию и инспекцию внутри команды, а для нескольких команд часто применяется масштабирование через фреймворки вроде SAFe, где события синхронизируют усилия без жесткой иерархии [10]. Такой механизм способствует высокой вовлеченности, но может привести к хаосу без сильного Scrum Master, особенно в распределенных командах, где ежедневные взаимодействия требуют дисциплины для поддержания фокуса на ценности продукта. V-Model, как последовательная методология, организует взаимодействие через иерархическую структуру, где левая сторона V фокусируется на декомпозиции требований с участием аналитиков и дизайнеров, а правая — на интеграции и верификации с тестерами, что подразумевает четкое разделение ролей без значительной самоорганизации, а координация между командами достигается через формальные документы и этапы валидации, делая подход подходящим для проектов с высокими требованиями к качеству, но ограничивая гибкость в динамичных средах [12]. Здесь взаимодействие больше напоминает водопадный стиль, с акцентом на планирование и контроль, где менеджеры координируют переходы между фазами, минимизируя риски через строгие протоколы, хотя это может замедлить обратную связь внутри команды. Rapid Application Development, или RAD, подчеркивает вовлеченность пользователей и быструю итеративность, где команды, включая разработчиков и конечных пользователей, взаимодействуют через joint application design сессии и прототипирование, без строгих ролей, но с фокусом на коллективном творчестве в малых группах, что способствует быстрому обмену идеями, а для нескольких команд подход масштабируется через параллельные циклы разработки, хотя отсутствие формальных митингов может требовать дополнительной дисциплины для синхронизации [14]. В сравнении с другими, RAD усиливает пользовательско-командное взаимодействие для ускорения, но может страдать от недостатка структуры в крупных проектах. Интересным дополнением к Scrum для оптимизации потока работ служит Kanban, который визуализирует задачи на досках с WIP-лимитами, управляя взаимодействием через ежедневные встречи и ретроспективы, где самоорганизация достигается путем явных политик и фокуса на потоке, позволяя командам, включая несколько, координировать усилия без фиксированных спринтов, что усиливает прозрачность и снижает перегрузки, особенно в комбинации со Scrum для гибридных подходов вроде Scrumban [16]. В целом, RUP и V-Model предлагают более иерархическое взаимодействие, подходящее для регулируемых сред, где роли четко разделены для контроля, в то время как OpenUP, Scrum и RAD продвигают самоорганизацию и коллективность, что повышает адаптивность, но требует зрелой культуры команды, а классификация по критериям взаимодействия показывает, что agile-подходы, такие как Scrum и OpenUP, лучше справляются с динамичными проектами за счет частых митингов и обратной связи, в отличие от традиционных, где акцент на документах и ролях [3]. Например, в Scrum и OpenUP ежедневные взаимо-

действия и ретроспективы способствуют непрерывному улучшению командной динамики, тогда как в RUP и V-Model обзоры привязаны к фазам, что может быть эффективнее для крупных межкомандных проектов с фокусом на рисках, а RAD выделяется пользовательским вовлечением, делая взаимодействие более инклюзивным, но менее структурированным [11]. В трендах гибкой разработки, как отмечается в обзорах, комбинации вроде Scrum с Kanban усиливают взаимодействие через визуализацию, позволяя командам лучше управлять потоком и сотрудничать в реальном времени, что особенно полезно для распределенных групп, где традиционные методологии вроде V-Model могут сталкиваться с задержками в коммуникации [2]. Таким образом, выбор организации взаимодействия зависит от размера команды, сложности проекта и нужды в гибкости: для малых самоорганизующихся групп подойдут Scrum или OpenUP, для структурированных enterprise-проектов — RUP, а для быстрых прототипов с пользовательским фидбеком — RAD, с возможностью интеграции Kanban для улучшения потока в agile-средах.

Заключение

В ходе проведенного сравнительного анализа методологий разработки программного обеспечения RUP, OpenUP, SCRUM, V-Model и RAD удалось выявить их ключевые особенности, сильные и слабые стороны, а также условия, при которых каждая из них демонстрирует наибольшую эффективность в управлении проектами. Общий вывод состоит в том, что выбор методологии напрямую зависит от характера проекта, его масштаба, требований к скорости реализации и уровня неопределенности в требованиях заказчика. Например, RUP, как итеративно-инкрементальная методология, ориентированная на управление рисками и архитектурой, показывает высокую эффективность в крупных проектах с четко определенными требованиями, где акцент делается на фазы inception, elaboration, construction и transition, но ее слабостью является избыточная бюрократия и необходимость в значительных ресурсах для документирования, что может замедлить процесс в динамичных условиях [6]. В отличие от этого, OpenUP, представляющий собой упрощенную версию RUP с фокусом на минимализм и сотрудничество, лучше подходит для средних команд, где требуется гибкость без излишней формальности, хотя его минусом остается зависимость от квалификации команды для самоорганизации [9]. SCRUM, с его принципами прозрачности, инспекции и адаптации, подчеркивает ценность клиента через спринты и ретроспективы, что делает его идеальным для проектов с изменяющимися требованиями, но здесь слабость проявляется в потенциальной хаотичности без строгого контроля, особенно в больших командах [10]. V-Model, с ее V-образной структурой, где левая сторона фокусируется на декомпозиции требований, а правая — на интеграции и верификации, обеспечивает высокий уровень качества за счет раннего тестирования, однако ее линейность делает ее уязвимой к изменениям требований на поздних этапах, что увеличивает риски переработки [12]. Наконец, RAD акцентирует скорость и вовлеченность пользователя через итеративное прототипирование, что позволяет быстро адаптироваться к отзывам, но ее недостатком является ограничение на длительность проекта — не более 60-90 дней — и риск снижения качества в угоду темпам [5]. Эти сильные и слабые стороны подчеркивают, что ни одна методология не является универсальной, и их "goodness" или степень соответствия целям, должна оцениваться через фреймворки вроде OPP, где объективы связываются с принципами и практиками для проверки адекватности, способности организации и эффективности [15]. В частности, сравнение показывает, что agile-подходы вроде SCRUM и OpenUP приоритизируют гибкость и клиентскую ценность, в то время как традиционные, такие как V-Model и RUP, ставят во главу угла качество и управление рисками, а RAD балансирует между скоростью и адаптивностью, но требует строгих временных рамок [14]. Выбор по критериям, таким как масштаб проекта, уровень рисков и тип команды, может быть структурирован с помощью многокритериальной классификации, где учитываются факторы вроде итеративности, документирования и взаимодействия, позволяя избежать субъективности в оценке [3]. Например, для проектов с высокими рисками и фиксированными требованиями предпочтительна V-Model, так как ее структура минимизирует ошибки на этапе интеграции, в то время как для стартапов с неопределенными требованиями лучше подойдет SCRUM, где ежедневные митинги и спринты обеспечивают быструю корректировку [4]. Рекомендации по применению методологий основаны на анализе

их адаптивности к современным трендам, где гибкость становится ключевым фактором успеха в динамичных бизнес-окружениях [2]. В частности, для крупных организаций с сложными системами, зависящими от жизненно важных процессов, таких как медицина или транспорт, целесообразно использовать RUP или V-Model, поскольку они предлагают формализованный подход с тщательным документированием и верификацией, что снижает риски и обеспечивает соответствие стандартам [5]. Однако в случаях, когда проект требует быстрого выхода на рынок и постоянного фидбека от пользователей, RAD окажется оптимальным, особенно если команда способна к итеративному прототипированию без глубокого предварительного планирования [4]. Для команд, ориентированных на agile-принципы, но нуждающихся в упрощенном процессе, OpenUP предоставляет баланс между структурой RUP и гибкостью, с акцентом на микро-шаги и checklists для тестирования, что делает его подходящим для средних проектов с распределенными командами [8]. SCRUM, в свою очередь, рекомендуется для инновационных проектов, где изменения требований — норма, и где роли Product Owner и Scrum Master помогают в управлении приоритетами, хотя для усиления потока работ его можно дополнить элементами Kanban, такими как визуализация задач на доске, чтобы избежать bottleneck'ов в процессах [16]. В целом, анализ подтверждает, что успех зависит не только от выбора методологии, но и от ее адаптации под конкретный контекст, включая квалификацию команды и организационную культуру, где оценка "goodness" через фреймворки вроде OPP позволяет выявить несоответствия на ранних стадиях [15]. Тренды указывают на растущую популярность гибридных подходов, сочетающих, например, итеративность SCRUM с верификацией V-Model, чтобы минимизировать слабости каждой [2]. Таким образом, перед внедрением рекомендуется использовать блок-схемы или матрицы сравнения для прогнозирования результатов, учитывая приоритеты вроде скорости, качества и взаимодействия, что в итоге повысит шансы на успешное завершение проекта [4]. В заключение, проведенное исследование подчеркивает необходимость индивидуального подхода к выбору методологии, опираясь на объективные критерии и реальные потребности проекта, чтобы избежать типичных ловушек, таких как переоценка гибкости или недооценка рисков [14].

Литература

1. RSDN Magazine. Современные процессы разработки программного обеспечения. 2006. URL: <https://rsdn.org/article/methodologies/softwaredevelopmentprocesses.xml> (дата обр. 12.01.2026).
2. Javarush Community. Сравнение методологий разработки ПО для начинающих. 2023. URL: <https://javarush.com/groups/posts/2443-vse-cto-nuzhno-znat-jh-o-metodologijakh-razrabotki-po-trendih-principih-i-lovushki-dlja-novichk> (дата обр. 12.01.2026).
3. “Методологии разработки программного обеспечения: анализ и классификация”. В: Перспективы науки = Science Prospects 6(177) (2024), с. 12—26.
4. Блок-схема выбора оптимальной методологии разработки ПО. 2023. URL: <https://habr.com/ru/amp/publications/297612/> (дата обр. 12.01.2026).
5. А. Д. Обухов и М. Н. Краснянский. Структурно-параметрический синтез адаптивных информационных систем на основе нейросетевых методов и архитектуры. Тамбов: ТГТУ, 2021, с. 5—8.
6. J. DeCarlo и др. The IBM Rational Unified Process for System z. IBM Redbooks. Armonk, NY: IBM International Technical Support Organization, 2007, с. 270. URL: <https://www.redbooks.ibm.com/redbooks/pdfs/sg247362.pdf> (дата обр. 12.01.2026).
7. Википедия. OpenUP. 2023. URL: <https://ru.wikipedia.org/wiki/OpenUP> (дата обр. 12.01.2026).
8. MyALM.ru. OpenUP: исследуем процесс. 2010. URL: <https://myalm.ru/news/OpenUP-%D0%B8%D1%81%D1%81%D0%BB%D0%B5%D0%B4%D1%83%D0%B5%D0%BC-%D0%BF%D1%80%D0%BE%D1%86%D0%B5%D1%81%D1%81> (дата обр. 12.01.2026).
9. Interface.ru. OpenUP — это просто. 2012. URL: <https://www.interface.ru/home.asp?artId=9995> (дата обр. 12.01.2026).
10. K. Schwaber и J. Sutherland. Scrum Guide 2020: Руководство по Scrum: официальное руководство. 2020. URL: <https://scrumguides.org/docs/scrumguide/v2020/2020-Scrum-Guide-Russian.pdf> (дата обр. 12.01.2026).
11. A.B.M. Moniruzzaman и S. Akhter Hossain. “Comparative Study on Agile software development methodologies”. В: arXiv (2013). eprint: 1307.3356. URL: <https://arxiv.org/abs/1307.3356>.
12. Wikipedia. V-Model. 2023. URL: <https://en.wikipedia.org/wiki/V-model> (дата обр. 12.01.2026).
13. Проектирование и разработка программного обеспечения : учебные материалы. 2023. URL: <https://xn--dlaux.xn--plai/proektirovanie-razrabotki-po/> (дата обр. 12.01.2026).
14. S. Nugroho и др. “Comparative Analysis of Software Development Methods between Parallel, V-Shaped and Iterative”. В: arXiv (2017). eprint: 1710.07014. URL: <https://arxiv.org/abs/1710.07014>.
15. S. Soundararajan и J. Arthur. “A Structured Framework for Assessing the «Goodness» of Agile Methods”. В: arXiv (2010). eprint: 1005.4123. URL: <https://arxiv.org/abs/1005.4123>.
16. Kanban University. The Official Kanban Guide: Руководство по методу Kanban. 2021. URL: https://kanban.university/wp-content/uploads/2021/11/The-Official-Kanban-Guide_Russian_A4.pdf (дата обр. 12.01.2026).